

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Diseño Web II

Entrega 6B

Mis Inscripciones y Retiro

FASE 3 — Inscripciones

Abril 2026

Proyecto: gestion-academica

Objetivos de Aprendizaje

Al completar esta entrega, el estudiante será capaz de:

- Crear una API Route PUT que actualiza el estado de un registro existente
- Implementar la lógica de retiro de inscripción con validación de estado previo
- Construir una página completa "Mis Inscripciones" con tabla y acciones
- Usar Badges semánticos para representar estados (inscrito = verde, retirado = rojo)
- Implementar filtros por estado de inscripción con Select
- Aplicar AlertDialog para confirmación de retiro (acción destructiva)
- Comprender la transición de estados unidireccional (inscrito → retirado)
- Integrar las vistas del catálogo (6A) y mis inscripciones (6B) como flujo completo

Prerrequisitos

Antes de comenzar, asegúrate de haber completado:

- ✓ Entregas 1A a 6A completadas y funcionando
- ✓ Al menos 4 inscripciones creadas desde el catálogo (Entrega 6A)
- ✓ Políticas RLS configuradas para inscripciones (Entrega 6A, Paso 4)
- ✓ La política UPDATE de inscripciones ya existe (la creamos en 6A)

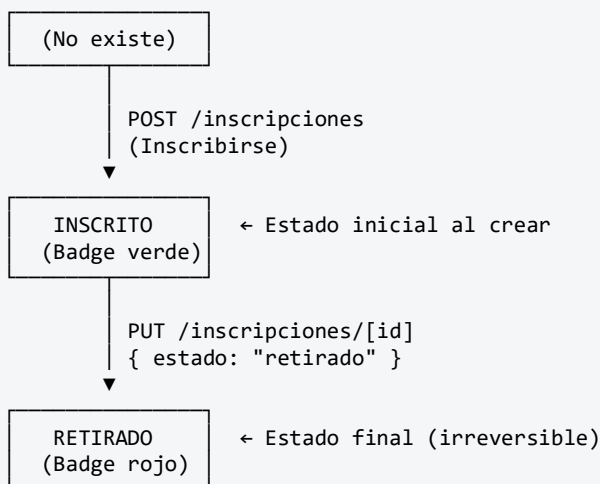
i Recordatorio: En la Entrega 6A ya creamos la política RLS "estudiantes_actualizar_sus_inscripciones" que permite UPDATE. La usaremos ahora para el retiro.

Teoría: Máquinas de Estado y Transiciones

Estado como Concepto de Diseño

En nuestra aplicación, cada inscripción tiene un campo "estado" que sigue reglas de transición específicas. Este concepto se conoce como máquina de estados finita (FSM):

Diagrama: Máquina de estados de inscripción



Reglas de Transición

De	A	Permitido	Método	Quién
(no existe)	inscrito	Sí	POST	Estudiante
inscrito	retirado	Sí	PUT	Estudiante
retirado	inscrito	NO	—	—
inscrito	(eliminado)	NO	—	—

⚠ Irreversible: El retiro es permanente en nuestro MVP. Una vez retirado, el registro permanece en la BD como evidencia histórica. El estudiante NO puede reinscribirse (UNIQUE constraint lo impide). En un sistema real, esto se manejaría con períodos de inscripción y semestres.

Soft Delete vs Hard Delete

Comparemos nuestro enfoque con la alternativa:

Enfoque	Operación	Dato en BD	Ventaja
Hard Delete	DELETE FROM inscripciones	Se borra por completo	Simple, permite reinscripción
Soft Delete (nuestro)	UPDATE estado = 'retirado'	Se conserva con estado	Historial, auditoría, reportes

☒ **Decisión:** Elegimos soft delete porque es más realista: en universidades reales, el retiro queda registrado en el historial académico del estudiante.

Paso 1 — API Route: Retirar Inscripción (PUT)

Este endpoint permite al estudiante cambiar el estado de su inscripción de "inscrito" a "retirado". Valida que la inscripción le pertenezca y que no esté ya retirada.

1.1 — Crear la estructura

Terminal

```
mkdir -p "app/api/estudiante/inscripciones/[id]"
```

1.2 — Crear la API Route

Crea el archivo `app/api/estudiante/inscripciones/[id]/route.ts`:

app/api/estudiante/inscripciones/[id]/route.ts

```
import { createClient } from "@lib/supabase/server";
import { NextRequest, NextResponse } from "next/server";

export async function PUT(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params;

    // 1. Verificar autenticación
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    // 2. Verificar que es estudiante
    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "estudiante") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    // 3. Obtener la inscripción actual
    const { data: inscripcion } = await supabase
      .from("inscripciones")
```

```

        .select("id, estado, estudiante_id")
        .eq("id", id)
        .single();

    if (!inscripcion) {
        return NextResponse.json(
            { error: "Inscripción no encontrada" },
            { status: 404 }
        );
    }

    // 4. Verificar que le pertenece
    if (inscripcion.estudiante_id !== user.id) {
        return NextResponse.json(
            { error: "No puedes modificar inscripciones de otro estudiante" },
            { status: 403 }
        );
    }

    // 5. Verificar que está inscrito (no ya retirado)
    if (inscripcion.estado !== "inscrito") {
        return NextResponse.json(
            { error: "Esta inscripción ya fue retirada" },
            { status: 400 }
        );
    }

    // 6. Actualizar estado a retirado
    const { data, error } = await supabase
        .from("inscripciones")
        .update({ estado: "retirado" })
        .eq("id", id)
        .select(`
            id, estado, created_at,
            materia:materias!materia_id(id, codigo, nombre)
        `)
        .single();

    if (error) {
        console.error("Error al retirar inscripción:", error);
        return NextResponse.json(
            { error: "Error al procesar el retiro" },
            { status: 500 }
        );
    }

    return NextResponse.json({ inscripcion: data });
} catch (err) {
    console.error("Error inesperado:", err);
    return NextResponse.json(
        { error: "Error interno del servidor" },
        { status: 500 }
    );
}
}

```

1.3 — Validaciones en cascada

Este endpoint tiene 4 niveles de validación antes de ejecutar la actualización:

Nivel	Validación	Error si falla	Status
1	Usuario autenticado	No autenticado	401
2	Rol es estudiante	No autorizado	403
3	Inscripción existe	No encontrada	404
4	Pertenece al estudiante	No puede modificar de otro	403
5	Estado es "inscrito"	Ya fue retirada	400

i Doble protección: La verificación de propiedad (paso 4) es redundante con RLS, pero es buena práctica tener protección explícita en el código. Si alguien desactiva RLS por error, el código sigue siendo seguro.

✓ Sin Service Role: Usamos el cliente normal que respeta RLS. La política UPDATE que creamos en la Entrega 6A permite que el estudiante actualice sus propias inscripciones.

Paso 2 — Componente: Mis Inscripciones

Este Client Component muestra todas las inscripciones del estudiante en una tabla con filtros por estado y acciones de retiro.

2.1 — Crear el componente

Crea el archivo `components/dashboard/estudiante/mis-inscripciones.tsx`:

components/dashboard/estudiante/mis-inscripciones.tsx — Parte 1/3

```
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";

import {
  Table, TableBody, TableCell, TableHead,
  TableHeader, TableRow,
} from "@components/ui/table";
import { Badge } from "@components/ui/badge";
import { Button } from "@components/ui/button";
import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue,
} from "@components/ui/select";
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel,
  AlertDialogContent, AlertDialogDescription,
  AlertDialogFooter, AlertDialogHeader,
  AlertDialogTitle, AlertDialogTrigger,
} from "@components/ui/alert-dialog";

import { RefreshCw, Logout } from "lucide-react";

interface Inscripcion {
  id: string;
  estado: "inscrito" | "retirado";
  created_at: string;
  materia: {
    id: string;
    codigo: string;
    nombre: string;
    credits: number;
    semestre: string;
    docente: {
      nombre: string;
      apellido: string;
    } | null;
  };
};
```


i Tipo completo: La interfaz Inscripcion refleja exactamente la estructura JSON que devuelve GET /api/estudiante/inscripciones, incluyendo el JOIN anidado materia → docente.

2.2 — Estado, carga y acción de retiro

components/dashboard/estudiante/mis-inscripciones.tsx – Parte 2/3

```
export default function MisInscripciones() {
  const [inscripciones, setInscripciones] = useState<Inscripcion[]>([]);
  const [loading, setLoading] = useState(true);
  const [filtroEstado, setFiltroEstado] = useState<string>("todos");
  const [retirando, setRetirando] = useState<string | null>(null);

  const cargarInscripciones = async () => {
    setLoading(true);
    try {
      const res = await fetch("/api/estudiante/inscripciones");
      const data = await res.json();
      if (!res.ok) {
        toast.error(data.error || "Error al cargar inscripciones");
        return;
      }
      setInscripciones(data.inscripciones);
    } catch {
      toast.error("Error de conexión");
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => { cargarInscripciones(); }, []);

  // Filtrar inscripciones
  const inscripcionesFiltradas = inscripciones.filter((i) => {
    return filtroEstado === "todos" || i.estado === filtroEstado;
  });

  // Contadores
  const totalInscritas = inscripciones.filter((i) => i.estado === "inscrito").length;
  const totalRetiradas = inscripciones.filter((i) => i.estado === "retirado").length;
  const totalCreditos = inscripciones
    .filter((i) => i.estado === "inscrito")
    .reduce((sum, i) => sum + (i.materia?.creditos || 0), 0);

  // Retirar inscripción
  const retirarInscripcion = async (inscripcion: Inscripcion) => {
    setRetirando(inscripcion.id);
    try {
      const res = await fetch(
        `/api/estudiante/inscripciones/${inscripcion.id}`,
        { method: "PUT" }
      );

      const data = await res.json();

      if (!res.ok) {
        toast.error(data.error || "Error al retirar inscripción");
        return;
      }

      toast.success(
        `Inscripción retirada: ${inscripcion.materia.codigo} - ${inscripcion.materia.nombre}`
      );
    }
  };
}
```

```
// Actualizar estado localmente (optimista)
setInscripciones((prev) =>
  prev.map((i) =>
    i.id === inscripcion.id ? { ...i, estado: "retirado" as const } : i
  )
);
} catch {
  toast.error("Error de conexión");
} finally {
  setRetirando(null);
}
};
```

☑ **Actualización optimista:** En lugar de recargar toda la lista, actualizamos solo el estado del registro modificado con `prev.map()`. El Badge cambia de verde a rojo instantáneamente.

2.3 — Renderizado: estadísticas, filtros y tabla

components/dashboard/estudiante/mis-inscripciones.tsx — Parte 3/3

```

if (loading) {
  return (
    <div className="flex items-center justify-center py-12">
      <p className="text-muted-foreground">Cargando inscripciones...</p>
    </div>
  );
}

if (inscripciones.length === 0) {
  return (
    <div className="flex flex-col items-center justify-center py-12 gap-2">
      <p className="text-muted-foreground">
        No tienes inscripciones aún.
      </p>
      <p className="text-sm text-muted-foreground">
        Ve al catálogo de materias para inscribirte.
      </p>
    </div>
  );
}

return (
  <div className="space-y-4">
    { /* Estadísticas rápidas */ }
    <div className="flex items-center gap-6 text-sm">
      <span>
        Inscritas: <strong className="text-green-600">{totalInscritas}</strong>
      </span>
      <span>
        Retiradas: <strong className="text-red-500">{totalRetiradas}</strong>
      </span>
      <span>
        Créditos activos: <strong>{totalCreditos}</strong>
      </span>
    </div>

    { /* Filtros */ }
    <div className="flex items-center gap-4">
      <Select value={filtroEstado} onChange={setFiltroEstado}>
        <SelectTrigger className="w-[200px]">
          <SelectValue placeholder="Filtrar por estado" />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="todos">Todos los estados</SelectItem>
          <SelectItem value="inscrito">Inscritas</SelectItem>
          <SelectItem value="retirado">Retiradas</SelectItem>
        </SelectContent>
      </Select>

      <Button variant="outline" size="sm" onClick={cargarInscripciones}>
        <RefreshCw className="mr-2 h-4 w-4" /> Refrescar
      </Button>

      <span className="text-sm text-muted-foreground ml-auto">
        {inscripcionesFiltradas.length} inscripción(es)
      </span>
    </div>
  </div>
);

```

```

{ /* Tabla */ }
<div className="rounded-md border">
  <Table>
    <TableHeader>
      <TableRow>
        <TableHead>Código</TableHead>
        <TableHead>Materia</TableHead>
        <TableHead>Créditos</TableHead>
        <TableHead>Semestre</TableHead>
        <TableHead>Docente</TableHead>
        <TableHead>Estado</TableHead>
        <TableHead>Fecha</TableHead>
        <TableHead className="text-right">Acción</TableHead>
      </TableRow>
    </TableHeader>
    <TableBody>
      {inscripcionesFiltradas.length === 0 ? (
        <TableRow>
          <TableCell colspan={8} className="text-center py-8">
            No se encontraron inscripciones
          </TableCell>
        </TableRow>
      ) : (
        inscripcionesFiltradas.map((i) => (
          <TableRow key={i.id}>
            <TableCell className="font-mono font-medium">
              {i.materia?.codigo}
            </TableCell>
            <TableCell>{i.materia?.nombre}</TableCell>
            <TableCell>
              <Badge variant="secondary">
                {i.materia?.creditos}
              </Badge>
            </TableCell>
            <TableCell>
              <Badge variant="outline">
                {i.materia?.semestre}
              </Badge>
            </TableCell>
            <TableCell>
              {i.materia?.docente
                ? `${i.materia.docente.nombre} ${i.materia.docente.apellido}`
                : <span className="text-muted-foreground italic">Sin asignar</span>
              }
            </TableCell>
            <TableCell>
              <Badge
                variant={i.estado === "inscrito" ? "default" : "destructive"}
              >
                {i.estado.charAt(0).toUpperCase() + i.estado.slice(1)}
              </Badge>
            </TableCell>
            <TableCell>
              {new Date(i.created_at).toLocaleDateString("es-B0")}
            </TableCell>
            <TableCell className="text-right">
              {i.estado === "inscrito" ? (
                <AlertDialog>
                  <AlertDialogTrigger asChild>
                    <Button
                      variant="destructive"
                      size="sm"
                      disabled={retirando === i.id}
                    >
                      <Logout className="mr-1 h-3 w-3" />

```

```

        {retirando === i.id ? "Retirando..." : "Retirar"}
      </Button>
    </AlertDialogTrigger>
    <AlertDialogContent>
      <AlertDialogHeader>
        <AlertDialogTitle>
          ¿Retirar inscripción?
        </AlertDialogTitle>
        <AlertDialogDescription>
          Estás a punto de retirarte de{" "}
          <strong>
            {i.materia?.codigo} - {i.materia?.nombre}
          </strong>
          . Esta acción es permanente y no podrás
          volver a inscribirte en esta materia.
        </AlertDialogDescription>
      </AlertDialogHeader>
      <AlertDialogFooter>
        <AlertDialogCancel>Cancelar</AlertDialogCancel>
        <AlertDialogAction>
          onClick={() => retirarInscripcion(i)}
          className="bg-destructive text-destructive-foreground hover:bg-
destructive/90"
        >
          Confirmar Retiro
        </AlertDialogAction>
      </AlertDialogFooter>
    </AlertDialogContent>
  </AlertDialog>
) : (
  <span className="text-sm text-muted-foreground italic">
    Retirada
  </span>
)
</TableCell>
</TableRow>
))
</TableBody>
</Table>
</div>
</div>
);
}

```

2.4 — Comportamiento de la columna Acción

Estado	Qué muestra	Interacción
inscrito	Botón rojo "Retirar"	Abre AlertDialog, ejecuta retiro al confirmar
retirado	Texto gris "Retirada"	Sin acción disponible

i Condicional en JSX: Usamos un operador ternario para mostrar el botón de retiro solo si el estado es "inscrito". Si ya está retirada, no tiene sentido mostrar un botón.

Paso 3 — Página: Mis Inscripciones

Reemplazaremos la página placeholder con la página real.

3.1 — Actualizar la página

Reemplaza completamente `app/(dashboard)/estudiante/inscripciones/page.tsx`:

```
app/(dashboard)/estudiante/inscripciones/page.tsx
import {
  Card, CardContent, CardDescription,
  CardHeader, CardTitle,
} from "@components/ui/card";
import { ClipboardList } from "lucide-react";
import MisInscripciones from "@components/dashboard/estudiante/mis-inscripciones";

export default function EstudianteInscripcionesPage() {
  return (
    <div className="flex flex-1 flex-col gap-4 p-4">
      <div className="flex items-center gap-3">
        <ClipboardList className="h-8 w-8 text-primary" />
        <div>
          <h1 className="text-2xl font-bold tracking-tight">
            Mis Inscripciones
          </h1>
          <p className="text-muted-foreground">
            Consulta y gestiona tus inscripciones en materias
          </p>
        </div>
      </div>

      <Card>
        <CardHeader>
          <CardTitle>Inscripciones Registradas</CardTitle>
          <CardDescription>
            Lista de todas las materias en las que te has inscrito
          </CardDescription>
        </CardHeader>
        <CardContent>
          <MisInscripciones />
        </CardContent>
      </Card>
    </div>
  );
}
```

☒ **Mismo patrón:** Server Component (Card + layout) + Client Component (tabla interactiva). Este patrón ya lo dominamos desde la Entrega 4A.

Paso 4 — Sincronizar Catálogo con Retiros

Cuando el estudiante retira una inscripción desde "Mis Inscripciones", el catálogo de materias también debe reflejar el cambio. El botón de esa materia debe cambiar de "Inscrito" a "Retirado".

4.1 — ¿Es necesario modificar el catálogo?

No directamente. El catálogo de la Entrega 6A ya maneja el estado "retirado": la función `renderBotonInscripcion()` muestra un botón "Retirado" con icono `XCircle` cuando el estado es "retirado".

Sin embargo, hay un detalle de UX: si el estudiante retira desde "Mis Inscripciones" y luego navega al catálogo sin recargar, los datos estarán desactualizados. Esto se resuelve naturalmente porque:

- Cada vez que el estudiante entra al catálogo, se ejecuta `cargarDatos()` en el `useEffect`
- La función carga las inscripciones frescas del servidor
- El botón se renderiza con el estado correcto ("retirado")

i Sin estado global: En nuestro MVP no usamos un estado global (como Zustand o Context). Cada página carga sus datos independientemente al montarse. Esto es simple y suficiente para nuestro caso.

4.2 — Flujo completo del estudiante

Con las entregas 6A y 6B completadas, el estudiante tiene este flujo:

Flujo completo del estudiante

1. `/estudiante/materias` (Catálogo)
 - Ver todas las materias disponibles
 - Filtrar por semestre
 - Clic "Inscribirme" → POST → Botón cambia a ✓
2. `/estudiante/inscripciones` (Mis Inscripciones)
 - Ver tabla con todas las inscripciones
 - Filtrar por estado (inscrito/retirado)
 - Ver estadísticas (inscritas, retiradas, créditos)
 - Clic "Retirar" → AlertDialog → PUT → Badge rojo

3. Volver al catálogo → materia muestra "Retirado"

Resumen de Archivos

En esta entrega creaste o modificaste los siguientes archivos:

Archivo	Acción	Descripción
app/api/estudiante/inscripciones/[id]/route.ts	Nuevo	PUT: retirar inscripción (estado → retirado)
components/dashboard/estudiante/mis-inscripciones.tsx	Nuevo	Tabla con filtros, badges, AlertDialog, estadísticas
app/(dashboard)/estudiante/inscripciones/page.tsx	Modificado	Reemplaza placeholder con MisInscripciones

Archivos sin cambios que se reutilizan:

Archivo	Reutilizado para
app/api/estudiante/inscripciones/route.ts	GET (listar inscripciones) y POST (inscribirse) siguen funcionando
components/dashboard/estudiante/catalogo-materias.tsx	Ya maneja estado "retirado" en el botón (Entrega 6A)
Supabase RLS: política UPDATE	Permite al estudiante actualizar sus inscripciones (creada en 6A)

Verificación Final

Ejecuta `npm run dev` y prueba con un usuario estudiante:

A. Página Mis Inscripciones

- ✓ Navegar a `/estudiante/inscripciones` desde el sidebar
- ✓ La tabla muestra todas las inscripciones con 8 columnas
- ✓ Las estadísticas muestran: inscritas (verde), retiradas (rojo), créditos activos

B. Badges de estado

- ✓ Inscripciones activas muestran Badge "Inscrito" en azul/default
- ✓ La fecha de inscripción se muestra en formato boliviano (dd/mm/aaaa)

C. Filtro por estado

- ✓ Filtrar por "Inscritas" muestra solo las activas
- ✓ Filtrar por "Retiradas" muestra solo las retiradas
- ✓ El contador se actualiza con los resultados filtrados

D. Retiro de inscripción

- ✓ Clic en "Retirar" abre `AlertDialog` con nombre de la materia
- ✓ El diálogo advierte que la acción es permanente
- ✓ El botón "Confirmar Retiro" es rojo (destructive)
- ✓ Cancelar cierra el diálogo sin hacer nada
- ✓ Confirmar cambia el Badge a "Retirado" (rojo) instantáneamente
- ✓ Aparece toast verde confirmando el retiro
- ✓ El botón "Retirar" desaparece y se muestra texto "Retirada"
- ✓ Las estadísticas se actualizan (una menos inscrita, una más retirada)

E. Protección contra doble retiro

- ✓ Intentar retirar una inscripción ya retirada devuelve error 400 (si se fuerza por API)

F. Sincronización con catálogo

- ✓ Después de retirar, navegar a /estudiante/materias
- ✓ La materia retirada muestra botón "Retirado" con icono rojo

G. Estado vacío

- ✓ Si no hay inscripciones, muestra mensaje con sugerencia de ir al catálogo

Ejercicio Propuesto

Practica lo aprendido con estas tareas adicionales:

1. StatsCards en la página: Reemplaza las estadísticas de texto por 3 componentes StatsCard (de la Entrega 4A) mostrando: Materias inscritas (icono BookOpen, color verde), Materias retiradas (icono XCircle, color rojo) y Créditos activos (icono GraduationCap, color azul).
2. Enlace al catálogo: Agrega un botón "Ir al Catálogo" que navegue a /estudiante/materias usando next/navigation useRouter(). Posiciónalo junto al botón Refrescar. También agrega un enlace inverso en el catálogo que lleve a Mis Inscripciones.
3. Exportar a texto: Crea una función que genere un resumen en texto plano de las inscripciones actuales y lo copie al portapapeles con navigator.clipboard.writeText(). Agrega un botón "Copiar resumen" que ejecute esta función y muestre toast de confirmación.
4. Fecha de retiro: Agrega una columna "Fecha retiro" a la tabla que muestre la fecha en que se realizó el retiro. Para esto, necesitarás agregar una columna "retirado_at" (TIMESTAMPTZ DEFAULT NULL) a la tabla inscripciones en Supabase, y actualizar el endpoint PUT para que la asigne al momento del retiro.

i Para entregar: Implementa al menos 2 de los 4 ejercicios. Toma capturas mostrando: la tabla con inscripciones mixtas (inscritas y retiradas), el diálogo de confirmación, y el catálogo reflejando el estado de retiro.

Próximo: Entrega 7A — Vistas Cruzadas

En la siguiente entrega crearemos las vistas que conectan roles entre sí:

- Vista docente: ver estudiantes inscritos en cada materia
- Vista admin: ver todas las inscripciones del sistema
- API Routes específicas para cada vista cruzada
- Activar el botón "Ver estudiantes" en la MateriaCard del docente
- Tabla de inscripciones para el admin con filtros por materia y estado

☒ **Preparación:** Asegúrate de tener: al menos 2 estudiantes con inscripciones en las mismas materias, inscripciones mixtas (algunas inscritas, algunas retiradas), y credenciales de admin, docente y estudiante para probar las 3 vistas.

Fin de la Entrega 6B